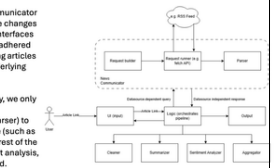
1.1 Software Application Deployment Process

Software Engineering is the application of a systemic, disciplined, quantifiable approach to the development, operation, and maintenance of software.



1.2 Edge Computing

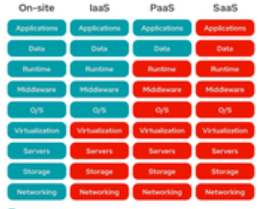
Edge Computing is a decentralized architecture that moves computational processing away from a centralized server (Cloud) to the "edge" of the network (near the data source). Meant to complement Cloud.

- **Edge**: for low latency & real-time responses → sensing, decision processes for self-driving vehicle.
- **Cloud**: for high power & big data ops → map updates, model training, storage for self-driving vehicle.
- BUT limited computation power on the edge (e.g., camera, sensor, or smartphone) and many of the devices at edge are not connected to a power source (issues of battery life and heat dissipation).

1.3 Cloud Native & Cloud Computing Apps

Cloud Computing's applications have their software infrastructure hosted on an external data center.

- Software/services delivered over internet
- Different models:
- **Internet-AAS**: AWS, Google cloud...
- **Platform-AAS**: AWS Elastic Beanstalk, Heroku...
- **Software-AAS**: Google apps, Salesforce...



Cloud-enabled: **Legacy applications** migrated to the cloud with minimal code changes → retains original features and structure → reduce infrastructure maintanence (get rid of physical servers) without cost of rewriting code BUT not fully leverage cloud benefits
vs Cloud-native: built/rebuilt specifically for the cloud.

- Characteristics: immutable infrastructure, microservices-based apps, API driven, service mesh, containers, dynamically managed.
- **Cloud-native development**: CI/CD, DevOps, serverless computing

1.4 Deployment Mechanism & Containerization

Bare Metal: app runs directly on physical hardware/host OS → Maximum performance (no virtualization overhead), physical isolation BUT low resource utilization (waste), scalability issues (**"dependency hell"**), low developer productivity

Virtual Machines (VMs): hardware virtualization via a **Hypervisor** (fully Guest OS running inside VM) → Better resource utilization than bare metal, flexible BUT heavy (CPU/RAM overhead from Guest OS), slow boot, **"Noisy Neighbour" problem** (high-load VMs can degrade performance of other VMs on the same host), **side-channel attacks** (exploit shared hardware to extract secrets by observing the implementation of other sys)

Containers: OS-level virtualization via **Container** Engine (e.g., Docker), has shared kernel instead of guest OS. → lighterweight (starts fast, low overhead), portable, no dependency issues (write once, run anywhere), granular (ideal for microservices & CI/CD) BUT slightly more overhead than bare metal.

Containerized Deployment: Packages Code + Runtime + Dependencies together → runs on heterogenous platforms and identical execution everywhere

- **Dockerfile**: text file acting as the "Environment Description" (recipe)
- **CI/CD**: good for automated pipelines due to speed and consistency
- **Container Orchestration** (e.g., Kubernetes): automated management of container clusters
  - Provisioning: place contains on avail servers
  - Scaling: auto-inc or decr replicas based on load
  - Availability: health checkes and self-healing
  - Networking: service discovery and load balancing between containers

1.5 Severless / FaaS (Function-as-a-service)

Cloud execution model where the provider manages the machine allocation entirely. NOT no server but no server management. use cases → burst traffic, background jobs (image processing), scheduled tasks (CRON jobs).

- Event-driven: code only runs in response to triggers (HTTP request, database change, file upload)
- Ephemeral: functions are short-lived (created → executed → destroyed)
- Stateless: no persistent local memory (must use external storage (DB/S3)
- Auto-Scaling: automatically scales fromn # of request
- Pay-as-you-go: metered execution time and memory (no idle cost → cost is zero when no requests)

1.6 Software Process & Scrum

Software Development Life Cycle (SDLC): Requirement Analysis → Design → Implementation → Testing → Evolution → .... (affected by : requirements, process resources, time, criticality, consequences, people (competence), technology)
Scrum Framework (Agile): iterative development with fixed-length loops (sprints).

- Roles: product owner, team, scrum master
- Artifacts: product backlog, sprint backlog, increment

1.7 Software Delivery

Deployment Pipeline: Automated implementation of application build, deploy, test, and release process.
Cont. Integration (CI): build → unit tests → deploy to stage → acceptance tests
Cont. Delivery: CI + MANUAL deploy to prod
Cont. Deployment: CI + AUTO deploy to prod
Cont. Deliver & Deployment: the ability to get changes of all types into prod, safely and quickly sustainable way.

1.7.1 Strategies for Continuous Deployment

- In-Place Deployment: stop instance → update code → restart. Simple and minimal infrastructure needed BUT downtime and risky (failed update breaks the only instance)
- Rolling Deployment: replace previous versions of the apps by completely replacing the infrastructure (e.g., containers running prev versions of the app is replaced 1-by-1 with containers running new vers). No downtime BUT no isolation (ver 1 & ver 2 coexist briefly), complex rollback (must reverse process)
- Blue/Green Deployment: two identical envs (blue=live v1, green=idle v2) deploy to green → switch router → green becomes live. Zero downtime, instant rollback so very safe BUT high cost (double infra needed)
- Canary Deployment: deploy new version to small subset → monitor errors → expand if safe. Lowest risk (if new ver fails simply remove and reroute back to original vers), supports A/B testing BUT complex traffic routing needed.

1.7.2 DevOps

DevOps is a set of SD practices that combine SD and ops to help reduce time between committing a change to a system, and the change being placed into prod.

- Practices: CI/CD, Continuous monitoring and logging, Communication and Collaboration, Infrastructure as Code.
- SRE Site Reliability Engineer: improving reliability of systems in production, fixing issues, responding to incidents, and taking on-call responsibilities

1.7.3 Recent Trends in Software Development Process

Pattern Detection builds on the theory that vulnerabilities follow repeatable patterns → use ML + expert knowledge to identify such problems
GenAI can boost productivity BUT:

- Garbage In Garbage Out → output is only as good as your input (prompt, code, etc.)
- Traceability → not all code can be used as is
- Data Safety → confidential data may be leaked
- Security Poisoning → bad actors can plant malicious code in training data
- Technical Debt → maintaining genAI code can b hard

2 Requirements - FR & NFR

FR specifies something that the system should do (usually a behaviour or function), NFR describes something not directly related to the functionality of the system (e.g., quality attributes). A combination of Software Quality Attributes define software quality:
External (observed when software is executing):

- Availability, Compatibility, Deployability, Integrity, Installability, Interoperability, Installability, Reliability, Robustness, Safety, Security, Usability,
- **Performance**: responsiveness of the system → Response Time, Throughput (how much data can be transmitted in fixed time), Data Capacity, Dynamic Capacity (max online users), Predictability for real-time systems, Latency (time delay), Behaviour in degraded models or overloaded conditions.

Internal (perceived by the developers and maintainers):

- Efficiency, Maintanability, Modifiability, Portability, Reusability, Testability, Verifiability,
- **Scalability**: ability to accomodate a growth in the application usage.
  - Vertical scaling is adding/upgrading capability of existing machine → easier to maintain BUT single point of failure
  - Horizontal scaling is increasing capacity by adding additional machiens (nodes) → increase resilience/fault tolerance BUT adds cost and complexity. (microservices designed to be small, focused and independently deployable → well suited for horizontal scaling)

3 Software Architecture

3.1 Introduction & Definitions

Core elements: **(1) Component**: element that models an application-specific function/responsibility/requirement/task/ process. **(2) Configuration**: topology or structure of components and connector. **(3) Connector**: element that models interactions among components for the purpose of transfer of control and/or data
Control Flow: chronological computation oreder (step A → step B)
Data Flow: focuses on data availability, transformation, latency (Input → Transformation → Output, downstream components activate when data arrives)
Call and Return: synchronous type, caller passes control to callee → callee works → control returns back to caller (can be hierarchical or non-hierarchical)
Message: each component has a unique address other components can send messages to → mainly synchronous: each recipient awaits messages and reacts to them.
Event: <already in other section>

3.2 Decomposition, Componentizing & Packaging

Horizontal slicing: designing by layers
Vertical slicing: designing by features

3.3 Principle of Modularity

Break complex problem down into smaller manageable modules (w well-defined interfaces, APIs)

3.4 Types of Cohesion

Functional: facilities perform only one computation with no side effects.
Layer: related services are kept together and there is a strict hieracrchy in which higher-level services can access only lower-level services.
Communicational: facilities for operating on the same data are kept together.
Sequential: a set of procedures, which are called one after another, are kept together.
Temporal: group procedures used in the same general phase of execution (initialization or termination)
Utility: related utilities are kept together (usu when no way to group them using a stronger form of cohesion)

3.5 Types of Coupling

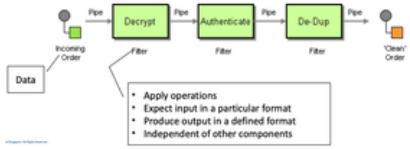
Content: component modifies internal data of anth [FIX: expose a clean interface (e.g., updateData function) instead of allowing direct access]
Common/Global: using global variables → all modules using the global variable are coupled. [FIX: replace global variable with a config manager that has read-only access]
Control: one module (A) directly controls the other (B) by passing information on what B should do. [FIX: replace the control flag w polymorphism/ED logic so B decides its action independently]
Data: one module sharing data with another module.
External: a dependency exists to elements outside the scope of the system, such as the operating system, shared libraries or the hardware.
Temporal: two actions are bundled together just because they happen to occur at the same time. [FIX: lazy initialization so the modules start only when they are needed]
Inclusion/Import: including tile or importing package

3.6 Architecture Styles

Technical Partitioning: focus on the seperation of concerns → may align with team's expertise.
Domain Partitioning: aligned with the domain → logical components map to problem. (e.g., customer, payment, ...)
Monolithic Deployment: deploy all logical components. that make up the application as one unit (runs as 1 proc)
Distributed Deployment: consists of independent logical components that run as individual processes → communicated over the network.

		Code partitioning	
		Technical	Domain
Code deployment	Monolith	E.g., Layered	E.g., Modular Monolith
	Distributed	E.g., Event-driven	E.g., Microservices

Layered Architecture: software is organized as layers of components → supports indendepent development and evolution of different system parts
Pipe and Filter: data enters the system and flow through the components one at a time until the data is assigned to some final dest (data sink). → good for processing of images, audio, video or batch data.



Model View Controller (MVC): seperate code into 3 components → **View** (UI widgets, buttons, text, ...), **Controller** (coordinates between M + V), **Model** (represents the business logic). Usu for apps with (G)UI

- V and C may be same object if tightly coupled
- Benefits: (1) Seperation of concerns → modularity (2) Extensibility → new view/controller pair can be added easily (3) Restricted communication → reduce complexity & side effects (4) Better testability (easy to mock components)

Web MVC: Server holds the model, Client interacts with the server (performs requests). **Controller** → handling user HTTP request, select the model. **View** → renders the HTTP (usu split for specific page and specific page requests, and **front controller**, for HTTP). **Model** → business logic + persistence (database).
Single Page Applications (SPA): web page that loads a single HTML entry point and dynamically updates content without full page reloads. V logic moves from server to client (browser). C requests data via AJAX/API, not HTML
Modular Monolith: Single deployable unit where the internal code is structured into loosely coupled, independent modules (Bounded Contexts)

4 REpresentational State Transfer, REST

REST is an architectural style for distributed systems (web APIs). Treats data as resources identified by Uniform Resource Identifier (URIs), manipulated via standard HTTP verbs (GET, PUT, POST, DELETE). **6 constraints of REST**:

- **Client-Server**: client not concerned with server related data and operations and vice versa

HTTP DESIGN: (table of) Operation (search for books by __), HTTP Request (GET), URI (/books/search?q={query})

- **Stateless**: interaction between C and S is entirely self-contained (no client state or "session" maintained on server) → server is bound by number of concurrent requests and not number of clients interacting
- **Cacheable**: responses explicitly label data as cacheable or non-cacheable → ↑ network efficiency
  Layered System: app must be organized as a (network) layered system (restricts complexity to individual layers) → intermediary servers can ↑ avail & perf.
- **Uniform Interface**: decouple implementa from service and ensure generalized interaction via standard HTTP.
  - resources are identified using stable, global and unique resource identifies (e.g., url params).
  - components perform operations on resource representations, not the raw resource.
  - self-describing messages (contain all parsing logic)
  - **hypermedia driving application state** (representations that are exchanged should be linked)
- **Code-On-Demand**: Server extends client functionality by sending code (e.g., JS).

REST allows systems to be less tightly coupled → ↑ scalability, ↑ usability, ↑ accessbility, ↑ mashup-ability BUT being stateless may decrease network performance by increasing the repetitive data sent, using URI may degrade efficiency since info transferred must be a standard form

5.1 Microservices

Microservices (MS) is an architectural style that structures an application as a collection of servcs that are organized around business capabilities, loosely coupled, owned by small team and independently deployable.

- **Service Instance per Host**: run each service instance in isolation on its host
- **Service Instance per Container**: each server instance runs in its own container (its own libraries etc.)
- **Service Instance per VM**: each service instance is a VM that is launched using that VM image

5.2 Domain Model

A system is fundamentally a collection of multiple domain models, sub-domains that can be **Core** (heart of biz), **Supporting** (related to core), **Generic** (not specific to biz)

5.3 Bounded Context

A bounded context is an explicit boundary within which a domain model exists.

5.3.1 Collaboration Between Bounded Contexts

Shared Kernel: two contexts share/overlap a subset of code or domain model → high coordination required, changes affect both.
Upstream-Downstream: direction of dependency, Provider → Consumer

- Supplier-Customer: Cooperative relationship → upstream team negotiates with downstream team to meet needs.
- Conformist: Uncooperative relationship → upstream provides a fixed API, downstream adapts.

5.3.2 Interaction Between Contexts

Model interactions on the real domain. Share only necessary data (e.g., warehouse needs only ItemID + Qty, not Customer profile). Communication patterns:

- **Synchronous:** REST/gRPC (HTTP) → simple but creates coupling (client waits)
- **Async**: event-based (pub/sub, queues) → decoupled

5.3.3 Aggregates

Cluster of related objects treated as a singlue unit for data changes

- Aggregate Root: parent entity (e.g., Ticket), aka the only object accessible from the outside.
- Aggregate Boundaries:
  - **Consistency** → external objects cannot modify internals directly, must call root methods
  - **Transactional** → any change to the aggregate succeeds or fails as a whole (atomicity)

5.3.4 Event Storming

Usu involves diff repres of Domain Events, Commands by diff stakeholders, Aggregate, Reactive Logic. Grouped together w boundaries → microservices.

RSS (really simple syndication): when a website publishes new content, auto add to its RSS feed file → users use an RSS reader to subscribe to specific feeds (like pub-sub but u check instead of they notify)

5.4 Data Patterns

Data Independence: concept that microservices should not modify the same data,.

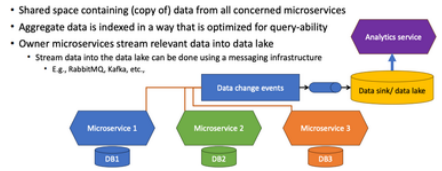
Database-server-per-service: each service has its own database server → loose coupling, scaling at database level, easy to replace underlying DB tech for ea service BUT expensive, more complex to manage diff DBs.,

Private-tables-per-service: each service owns a set of tables that must be accessed by that service.

Schema-per-service: each service has a database schema that is private to that service.

Shared database (antipattern): tightly coupled, ACID properties should be enforced.

Data Lake Pattern: read-only, queryable data sink.



Saga Pattern: every step of a transaction defines a compensating action (which executes if we need to roll back due to a failure). **Compensation action** is registered on a **routing slip** and passed along the next step → If one of the later step fails, it kicks off execution of all compensating actions on the routing slip → system is brought back to a reasonably compensated state. (Saga IS NOT ACID transaction → NOT undo)

5.4.1 Event Sourcing and CQRS

Events (i.e., state changes) are added to an immutable append-only log → Auditable (provides a complete history of every action), Able to rewind and reply events to reconstruct system state at any point, Easy to debug why a system reached a bad state.

Command Query Responsibility Segregation (CQRS) involves seperation of commands (write/update operations) from queries (read operations) because writes and reads have different performance, scaling and consistency requirements. (e.g., **Kafka**: Write → Kafka (event log) → Stream Processor (KSQL) → Update View → Read).



Benefits:

- scale read and write infra seperately
- create specific tables for specific screens
- internal state can be reconstructed from event log if the view DB crashes

5.5 Other Patterns

API Gateway: Single entry point server that encapsulates internal architecture → routes requests to appropiate microservices

Service Discovery: Database of active services and locations (e.g., Zookeeper, Consul) because dynamic IPs in cloud environments make hardcoding addresses impossible.

- **Client-side**: client queries reistry → client selects instance (load balances) → client calls service
- **Server-side**: client calls load balancer → LB queries registry → LB routes to service

Service Communication

- **Synchronous**: request/response (REST). Client waiting and high coupling.
- **Asynchronous**: one way notification (messaging). Client fires and forgets, low coupling.

Service Collaboration

- Orchestration: central "coordinator" service directs the flow → good for visibility BUT tight coupling.
- Choreography: decentralized, services react to events → loose coupling BUT harder to track flow.

6 Event-Driven Architecture (EDA)

6.1 Event Types

- Event: something important happened in the business/system
- Intent is to announce something has happened
- Initiating event: from end-user
- Derived event: internal event triggered by another event

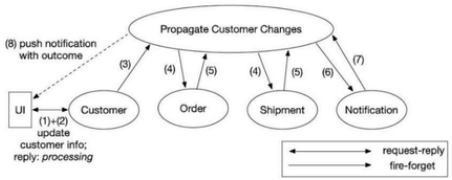
6.2 Event Structure

- Event = key–value pair
- Key: ID for routing/aggregation
- Value: full event details

6.3 Asynchronous Communication Introduction

6.3.1 Async Pros & Cons

- Better responsiveness, Better availability BUT more complex/fragmented error handling than in sync



6.4 Event-Driven Architecture (EDA concept)

- EDA: decoupled components communicate async via events (event based + async communication)
- Components: Producers, Broker, Consumers

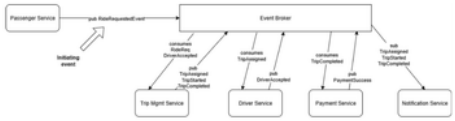
Producer: generates events

Broker: event bus → receives, stores, publishes on topics, enables subscriptions

Consumer: listens/consumes events

6.5 Pub/sub system (EDA Implementation)

- Producers = publishers → send events to topics
- Broker (Kafka) → manages topics + partitions events
- Consumers = subscribers → read from topics



6.6 EDA vs Microservices architecture

Communication & Performance

- **EDA**: async → fast responses
- **Microservices**: sync REST → latency

Bounded Contexts & Ownership

- **EDA**: optional
- **Microservices**: required (strong cohesion)

Event vs Request

- **EDA**: react to events (happened) → trigger more events
- **Microservices**: handle requests/commands (to happen)

6.7 Event-driven microservices (EDA + μservice)

6.7.1 Characteristics: Async Communication, Cohesive Service → ensures single responsibility principle (SRP), Data Ownership

6.7.2 Event-driven microservices structure:

- Event stream: ordered events stored in broker
- One producer per stream (owns that event type)
- Producer μservice: write events to streams
- Consumer μservice: read/process events from one or more streams

6.8 Event Broker

- Stores incoming events (queue/partitions)
- Provides events for consumers to read

6.8.1 Data flow (producer → broker → consumer)

- Producers write events to partitions (indexed queue that persists after pop)
- Each partition = append-only event log
- Load-balanced consumers share partitions (Kafka consumer groups)
- Rate limits apply to producers, consumers, and consumer groups

6.8.2 Event broker Properties

- Immutable: events never change
- Ordering: partition preserves publish order
- Indexing: each event has offset for seeking
- Partitioning: stream split into sub-streams
- Infinite retention: keep all events → rebuild state
- Replayability: replay events → source of truth

6.8.3 Scalability achieved with event brokers

- Parallel read/write across partitions on machines
- Scale out by adding machines → broker rebalances partitions

6.9 Pros of Event-Driven Microservices

- Performance: async + parallel = fast
- Scalability: scale services independently
- Maintainability: loosely coupled services
- Evolvability: add new consumers to events
- Fault tolerance: one service fails → others unaffected
- Testability: easy to mock due to low dependencies

6.10 Cons of Event-Driven Microservices

- Low testability: async + many services = hard to test
- High complexity: async flows + multiple DB topologies/tradeoffs

6.11 EDA summary

- Events: trigger services + replicate/copy data
- Event-based communication decreases coupling
- Event-driven microservice is a small application built to fulfil a specific bounded context.

7 Asynchronous Communication

Sync (HTTP):

- Tightly coupled, connection overhead
- Sender blocks + must handle failures

Async (AMQP):

- Loosely coupled, no blocking

Needs notifications + context mapping for replies

7.1 Asynchronicity in Request-Reply Pattern

- API queues work, returns 202 + status URL
- Worker proccess job, stores result in shared storage
- Status endpoint which client polls:
- done → return result/redirect
- pending → return 202 + ETA

7.2 Event vs Message

Message is directed (specific sender and receiver) but Event no specific receiver (broadcast nature, multiple subscribers may react).

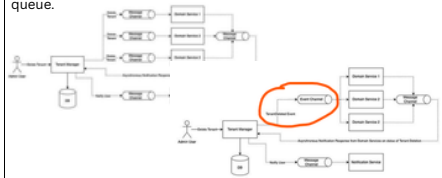
7.2.2 Event Pattern (Kafka, an event broker, supports via Topic constructs)

- Fact: state change broadcast (pub-sub)
- 1 → many, parallel via partitions
- Publisher owns event + topic

7.2.3 Message Pattern (AWS SQS, a message broker, supports this pattern through Queue construct

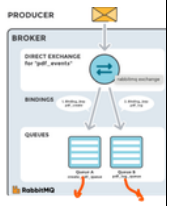
- Command/query to another service
- 1 → 1, FIFO, sequential
- Receiver owns message + queue

For Parallel work use event pub sub, then aggregate back later in queue.



7.3 Advanced Message Queue Protocol (AMQP)

- P2P async protocol via broker (e.g., RabbitMQ)
- Client publishes to exchange
- Exchange routes to queues
- Consumers either pull messages from queue or broker pushes to consumers.



7.3.1 AMQP implementation (Rabbit MQ) & Properties

- Supports async persistent messaging
- Sender writes messages into queues
- Guarantee: message will eventually. reach receiver's queue
- No guarantee on when/if receiver reads it

7.4 Persistent vs Transient Communication

- Persistent Communication : store-and-forward delivery, messages are stored until next node receives (e.g. email systems)
- Transient Communication: messages are buffered for some time (e.g. TCP communication)

7.5 Persistence & Synchronicity

Persistent Async: non-blocking; guaranteed delivery; stored durably; receiver can be offline.

Transient Async: non-blocking; no persistence (may lose msg); receiver must be online; lightweight.

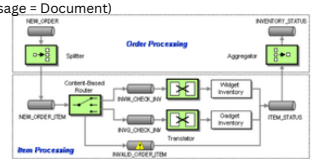
Persistent Sync: sender blocks until receipt ACK; msg persisted while waiting; slow but not lost.

Transient Sync (3 types):

- Receipt-based: block until ACK.
- Delivery-based: block until delivered (async RPC).
- Response-based: block until full reply (RPC).

8 Messaging

- (every D.A.D needs a M.O.M) D.A.D: distributed apps needing communication (e.g., microservices)
- M.O.M: messaging middleware (brokers) that manage + decouple communication (e.g., RabbitMQ/AMQP)
- EIP: Enterprise Integration Patterns is vendor-independent messaging patterns; same concepts, different names (e.g., Message = Document)



8.1 General Message Structure

- Message = Header + Properties + Body (or Metadata + Data)
- Header: type, origin, destination, size, structural info
- Properties (optional):
  - app-related (e.g., role)
  - provider-related (broker/queue info)
  - standard (TTL, priority)
- **Body**: actual payload (command/data)

8.2 General Message Channel

- Connects senders ⇔ receivers for message exchange
- Messages are persisted even if either side is offline

8.3 Request/ Reply Message Channels (2 chnls)

- **Requestor** sends request message through **Request Queue** to Replier.
- **Replier** receives the request message and responds with a reply message through the **Reply Queue**
  - Accordingly a 2 way message requires both the Request & Reply Queues.

8.3.1 Request/ Reply Message Structure

- ∃ Return address: Tells replier which queue to send reply to. Will be null value in reply.
- ∃ Message/ Request ID: Unique identifier
- ∃ Correlation ID: Specifies which request this reply is for → Send request message ID, becomes the Correlation ID of the sent reply. This is how matching is done to know which response matches to which request. Will be null value in sender

8.3.2 Request/ Reply Resolving

- Correlation Identifier: The process of matching requests with replies. Utilizing the Request ID and Correlation ID stated above.
- Request-Reply Chaining: Chain correlation ID to message IDs from the latest reply back to the original request (e.g. for auditing)

8.4 Special Message Channels

- Invalid Message Channel: holds bad/erroneous messages
- Dead Letter Channel: holds messages that couldn't be delivered/processed
- Data Type Channel: channel contains one data type, so receiver knows the format (e.g., RabbitMQ direct exchange routes by type)

8.5 Publish-Subscribe Message Channel (one)

- Broadcast message to all interested subscribers
- Multiple parties receive same message (e.g., MQ Topic)

8.6 Publish-Subscribe Message Channel (two way request and response pattern)

- Send 1 request → many subscribers (topic)
- Each service works in parallel
- App aggregates all replies into one result
- Services send replies to response queue

8.7.1 Message Routing - Types

Message Router: Component that decides destination of a message (your code routes msg → channel).

Simple Router: 1 input → 1 of many outputs (single destination only).

Composed Router: Chain of simple routers.

Content-Based Router: Route by message content; needs recipient knowledge; decouples sender.

Message Filter: Special content-based router; drops unwanted messages; single output.

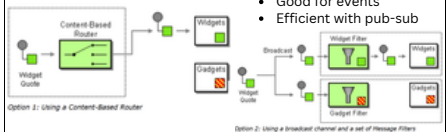
Context-Based Router: Route by metadata/environment, not message body:



8.7.2: Content-Based Router vs Pub-Sub + Message Filter

- Central routing component controls routing logic
- Router config must update to include new route or condition
- Good for business transactions
- Efficient with queues

- Subscribers definite its own filter, routing is reactive
- New subscriber just subscribes & apply filter
- Good for events
- Efficient with pub-sub

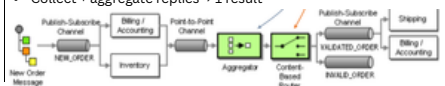


8.7.3 Message splitting and aggregating (not routers)

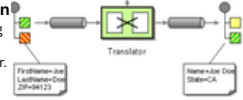
- **Not routers** (no routing decision).
- **Splitter:** 1 → many messages.
- **Aggregator**: many correlated → 1 message.

8.7.4 Message Scatter-Gather Pattern

- Broadcast 1 → many participants
- Collect + aggregate replies → 1 result



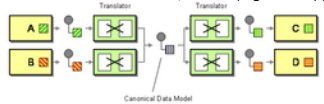
8.8 Message Transformation



- Msg Translator: Converts msg format so components can communicate with each other. Reduces coupling
- Many translators = O(n²) mappings → hard to maintain

Canonical Data model

- One common format in middle
- Each service has 2 translators (in/out) → O(n)
- Much easier to maintain; avoids spaghetti mappings



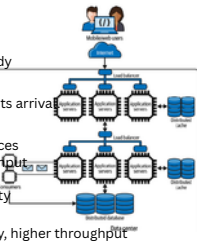
8.9 Message Endpoints

Encapsulates Messaging APIs. Message Endpoint:

- Interface between app ⇔ messaging system
- Convert data → msg (sender) / msg → data (receiver)
- Hide messaging details from app
- Separate sender/receiver
- 1 endpoint per channel

8.10 Polling Consumer

- Consumer pulls messages when ready



8.11 Event Driven Consumer

- Reactively processes a message on its arrival

9 Scalability

- Handle more work by adding resources
- Measured via response time/throughput
- Scale up or down based on load
- Add servers/CPU to increase capacity

9.1 Strategies to scale

- Replicate resources → more capacity, higher throughput
- Optimize resources → faster algorithms, indexing, faster languages

9.2 Scaling Post-Hoc is expensive

- Adding capacity later = downtime + expensive
- Often requires DB redesign + server rewrite
- Scalability must be planned from the start

9.3 System Architecture & Evolution

- System starts as monoliths, as load grows → latency → bottleneck. Single server is insufficient.

9.3.1 Scale Up (Vertical)

- Bigger server (more CPU/RAM) → simple, handles more load BUT physical limits, expensive

9.3.2 Scale Out (Horizontal)

- Multiple replicas behind load balancer
- Needs LB + session store (user in different replicas)

LB: picks a replica for each request

Session store: keeps user session shared across replicas → More capacity (add replicas), Resilient to instance failure BUT Single DB bottleneck → use cache /scale-out DB

9.3.3 Caching (Redis) - reduce DB querying

- Keep hot data in memory
- Read cache → DB on miss
- Prevent stale data via invalidation/clearing

9.3.4 Scaling Databases

Scale Up (vertical)

- Easy but: single point of failure, global latency, huge DB slows indexing → need distributed DB.

Scale Out – Read Replicas

- Primary = writes, replicas = reads
- Async replication; place replicas near global users.

Scale Out – Partitioning

- Horizontal: split rows (e.g., region='Asia')
- Vertical: split columns (coln 1)

Distributed Databases

- DB across multiple nodes
- Unified logical view retained + higher availability

9.3.5 Scaling Processing with multiple tiers

- Layers: UI → App → Core → DB
- Each tier LB + cached
- Tiers scale independently
- Services call replicated services
- Core tier handles DB
- Supports BFF (backend per frontend)

Advantages:

1. Responsiveness
   - Cache + async queue → fast replies
   - Queue writes faster than DB writes
2. Scalability
   - Tiers + BFF scale independently
3. Fault Isolation (Swim Lanes)
   - System split into isolated lanes
   - Failures contained; region-specific scaling; faster client responses

9.4 Scale Cube (X/Y/Z)

A model that explains ALL the ways a system can scale

- X-axis: replicate app (horizontal scaling)
- Y-axis: split by services (functional decomposition)
- Z-axis: shard data (segments like region/customer)